Prop Probability Engine

IN PROGRESS

A probability tool, not a betting tool.

Project brief by Vel'Shaun Byers

PLANNED STACK React, FastAPI, Python, LLM reasoning layer, sports data APIs, OAuth 2.0, pytest, GitHub Actions, Vercel, Railway

1. Why I Am Building This

I had a conversation that led me to believe the house is not supposed to always win. I am a strong believer in numbers and probability. I felt that sportsbook numbers are the reason the house always wins, and that those numbers do not reflect actual probability.

So I wanted to create a check and balance. Something that gives a better understanding of what people are actually betting on.

I am not a bettor. I just felt challenged to see if I could figure this out. That is literally my why.

2. What I Hope to Achieve

Friends and family are my beta group. They will use the tool and give me their unbiased opinion on whether they had more success with it than without it. I am using a small number of people to get enough data to decide if I move forward, or if the house always wins.

- It returns a probability for a player prop, over or under, with the factors that drove the number.
- It shows sample size, so a strong looking number built on three games is not mistaken for a sure thing.
- It breaks down matchups by position and role, such as what a defense allows to a second receiver compared to a first.
- It reads a screenshot of a slip, pulls out each prop, and ranks them by confidence.
- It is analytical only. Probabilities and evidence, no opinions or hunches.

The tool does not place bets or handle money.

3. Integrations

- **Player and team statistics.** A sports data API for game logs and defensive splits.
- **Posted lines.** Current lines to compare against. I am interested in PrizePicks and Kalshi as sources, where their terms and APIs allow it.

I am starting on the free tiers of these APIs on purpose. If the tool proves successful with the beta group, I plan to enhance it by moving to the paid tiers for deeper and faster data.

4. AI Integrations

An LLM reasoning layer takes the data and returns the probability together with its reasoning, so the user gets complete insight and not just a number. The same layer reads the slip screenshots.

The provider sits behind one interface so it can be swapped. I am starting with whichever free tier model does the job well, and I can change it later without rewriting the app.

5. CI/CD Pipeline

Every push and pull request runs through GitHub Actions [1]. The workflow installs dependencies, runs the test suite, and builds the front end.

The React front end deploys to Vercel and the FastAPI back end deploys to Railway. A change only reaches the live site after the checks pass.

6. Testing

Back end tests are written with pytest [2] and run in the pipeline on every push.

The first tests cover the probability calculations and the code that parses API responses and slip screenshots, since a bad parse quietly produces a bad number.

7. Security

- Sign in uses OAuth 2.0 [3], so the app never stores a password.
- I use the OWASP Top Ten [4] as my checklist while building and reviewing.
- API keys for the data and AI providers stay on the server in environment variables. They are never sent to the browser or committed to the repository.
- Uploaded screenshots are used to extract the props and are visible only to the user who uploaded them.

8. Accessibility

I build to the federal standard, Section 508. The Revised 508 Standards incorporate WCAG 2.0 Level AA by reference, so that is the bar I test against [5] [6].

- Every feature works with a keyboard alone, with a visible focus indicator.
- Text and controls meet the Level AA contrast ratios.
- Form fields have labels, and images and icons have text alternatives.
- Information is never carried by color alone.

9. Versioning and Lifecycle

I follow Semantic Versioning [7]. A bug fix bumps the patch number, a new feature bumps the minor number, and a breaking change bumps the major number.

Every release is tagged in GitHub with a short changelog entry. I am the only developer, but the rule does not change. It just means I am the one who enforces it.

Planned enhancements.

- Paid data tiers, if the beta results justify the cost.
- Response caching, once the beta group grows past the free API limits. With a small group of friends I do not expect to hit those limits yet.

> **Current status.** In progress. Data integrations come first, then the interface, then the friends and family beta.

References

[1] GitHub. GitHub Actions documentation. https://docs.github.com/en/actions

[2] pytest documentation. https://docs.pytest.org/

[3] Internet Engineering Task Force. RFC 6749, The OAuth 2.0 Authorization Framework. https://datatracker.ietf.org/doc/html/rfc6749

[4] OWASP Foundation. OWASP Top Ten. https://owasp.org/www-project-top-ten/

[5] Section508.gov. Applicability and Conformance Requirements, Revised 508 Standards. https://www.section508.gov/develop/applicability-conformance/

[6] World Wide Web Consortium (W3C). Web Content Accessibility Guidelines (WCAG) 2.0. https://www.w3.org/TR/WCAG20/

[7] Semantic Versioning 2.0.0. https://semver.org/